

Lecture 15: Virtual Memory

Professor G. Sandoval

15



From last Memory class

In the beginning... no abstractions

Downsides:

- Can't really have two independent programs running at the same time
- Each would have to know explicitly about what memory was in use by the other – requiring cooperation
- Note that it is possible to have multiple threads with flat memory. Why?

Lect 15: A process tries to access memory address 0x1234. Another process also uses address 0x1234. Without any memory abstraction, what happens?

Each process gets its own copy of that data -> no conflict

0%

Both processes access the same physical memory location -> potential corruption

0%

The OS automatically redirects one process to a different address

0%

A segmentation fault occurs immediately

0%

None of the above.

0%

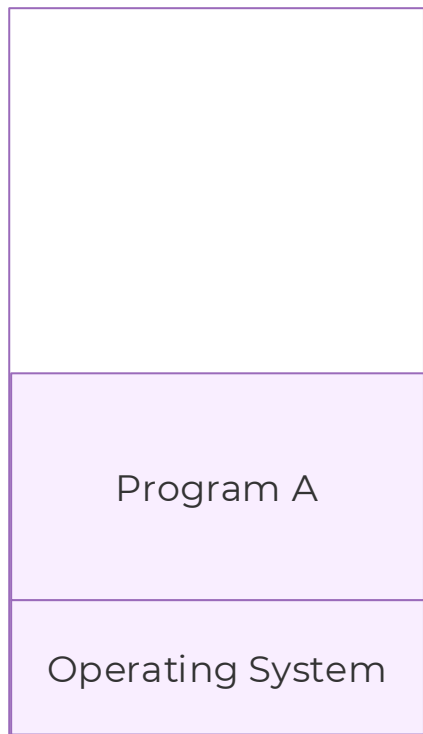
Swapping

- We may not have enough RAM to keep all the programs we're running in memory at once
- One strategy to get around this is to **move** programs from memory to disk when they're not being used (*swapping*)

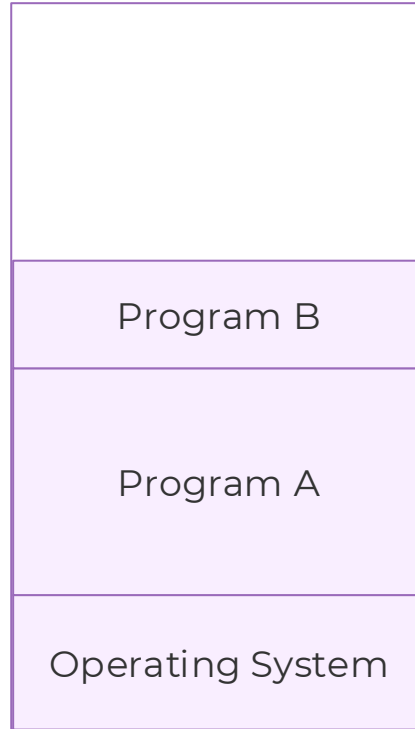
Swapping



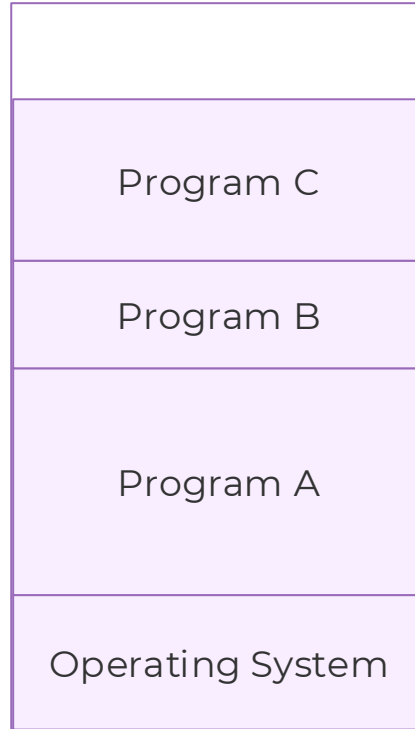
Swapping



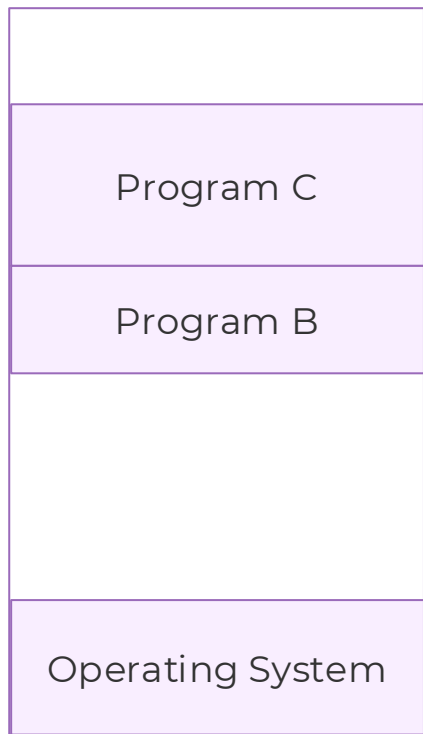
Swapping



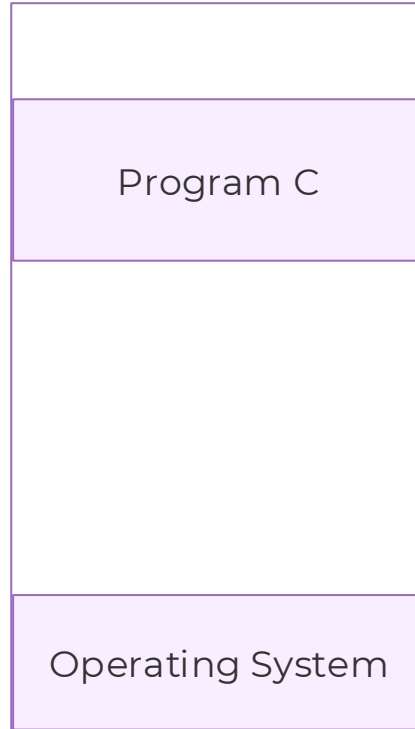
Swapping



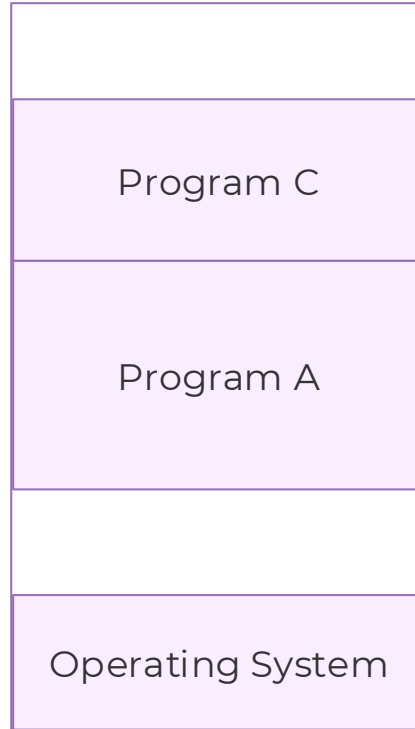
Swapping



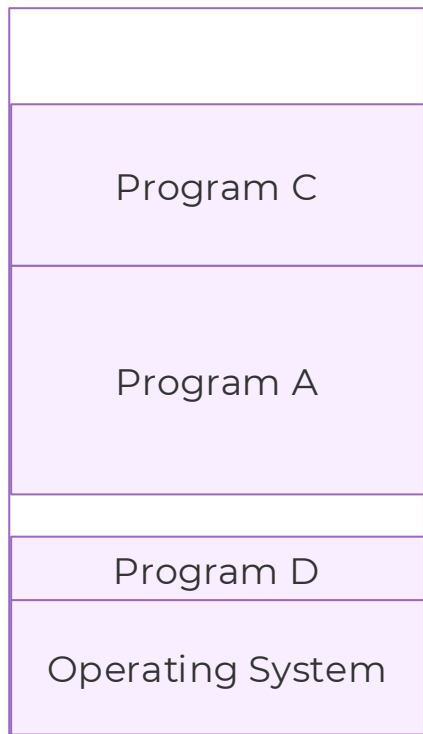
Swapping



Swapping



Swapping



Memory Compaction

- As a consequence of swapping things in and out of memory, we might *fragment* memory
- This could prevent us from loading a program even though we technically have enough memory for it
- If necessary, we can shuffle things around so that we have one contiguous free space instead of multiple small "holes"
- But: it may be *slow*! E.g. if it takes us 100 ns to read and then write 8 bytes of memory, *~100 seconds* to move 8GB

Lect 15: After several processes load and exit, you have 30KB of free memory total but cannot load a 20KB process. What is the most likely cause

The OS bitmap is corrupted

0%

The memory is fragmented into small non-contiguous holes

0%

The process is too large for the system

0%

Swapping has not been enabled

0%

None of the above

0%



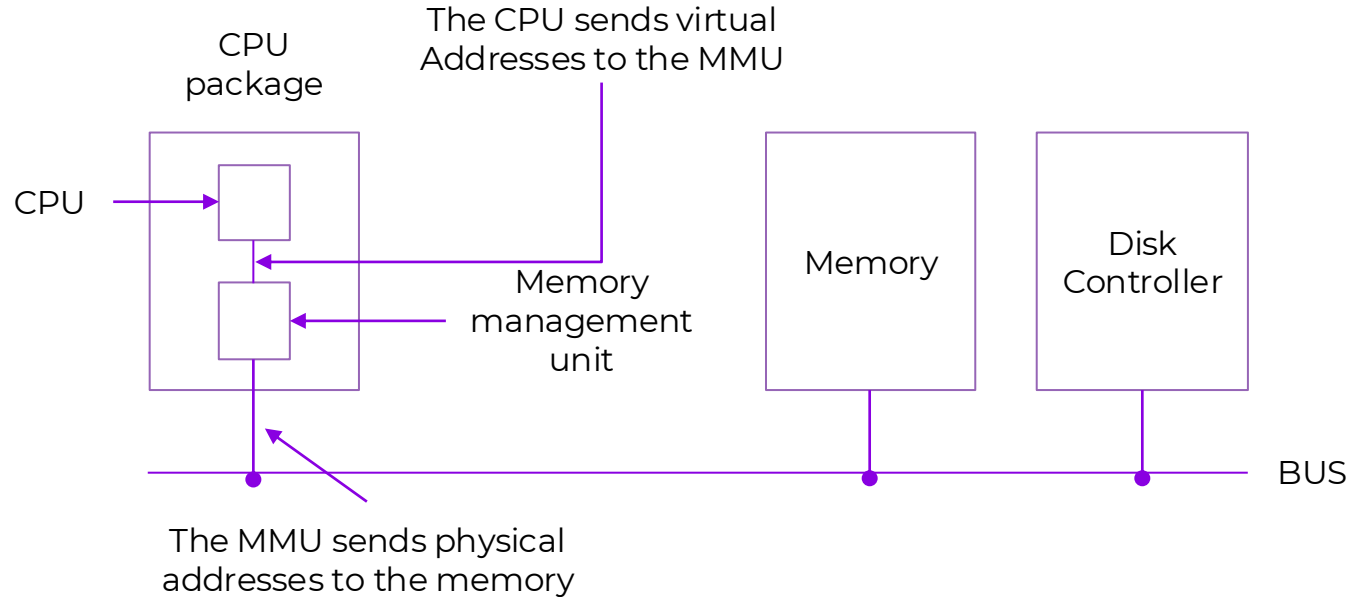
Virtual Memory



Virtual Memory

- Recall from last time – segmentation is no longer used to separate processes' memory from one another
- Instead, *virtual addressing* is used
- Each memory access no longer refers directly to physical memory, but instead is mapped to some actual physical address

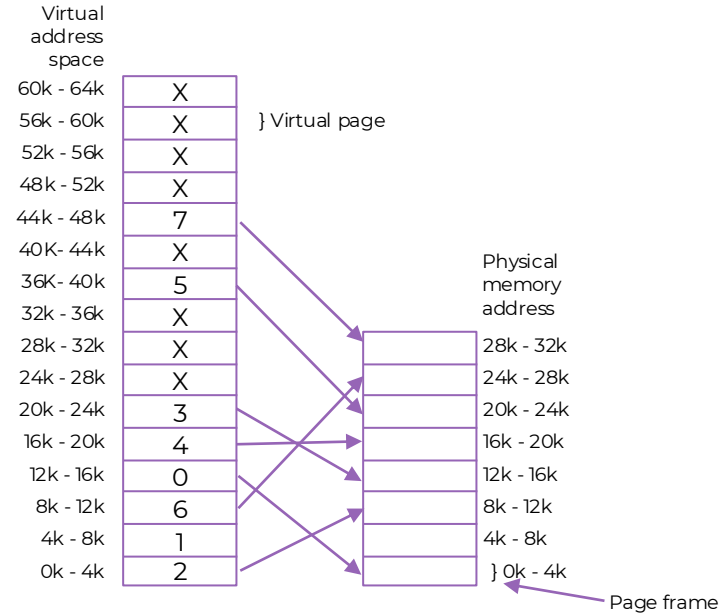
Virtual Memory



Paging

- Virtual memory is broken up into fixed-sized units (commonly, 0x1000 bytes (4096 decimal)) called *pages*
- Page sizes can vary though:
- 32-bit x86 supports 4KB and 4MB pages
- 64-bit x86 supports 4KB, 2MB, and 1GB pages
- The underlying physical pages of memory are called *page frames*

Paging



The relation between virtual addresses and physical memory addresses is given by the page table. Every page begins on a multiple of 4096 and ends 4095 addresses higher, so 4K–8K really means 4096–8191 and 8K to 12K means 8192–12287

Page Faults

- What happens if we try to access a page that is not mapped?
- The MMU notices, and we raise a CPU exception called a *page fault*
- Control is passed to the OS (via the usual interrupt/exception handling mechanism) to decide what to do
- Kill the process (*segmentation fault*)
- Find some physical page to map to it

Programs Bigger than Memory

- Note that this gives us a way to have programs that don't all fit into memory at once
- We can just **map** in the parts of the program we're using right now
- If we hit code or data that **isn't mapped**, we can **swap some other page** to disk and update the mappings

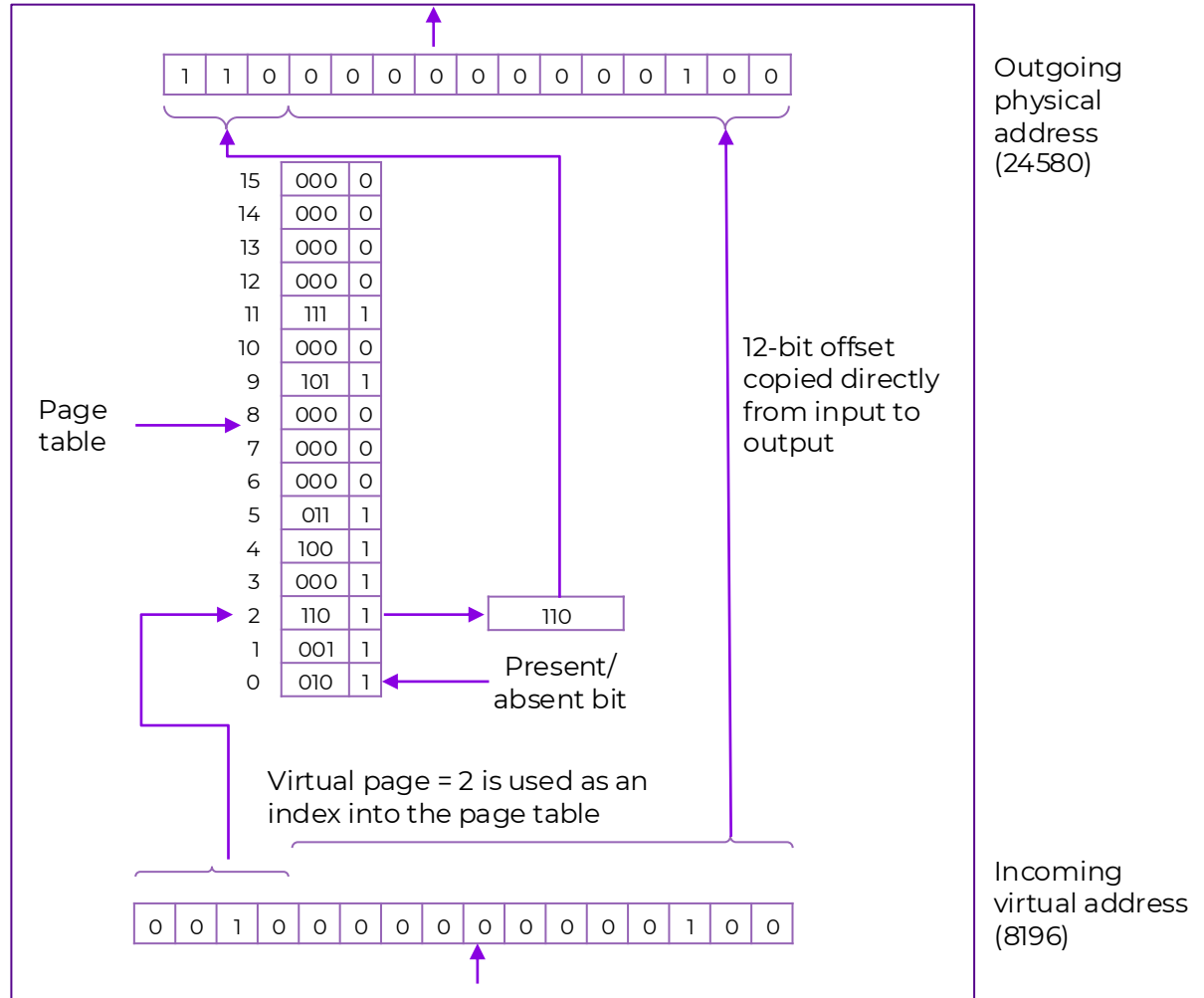
Types of Page Fault

- Minor page fault – can be serviced by just creating the right mapping
- Major page fault – must load in a page from disk to service
- Segmentation fault – invalid address accessed; can't service so we usually just kill the program

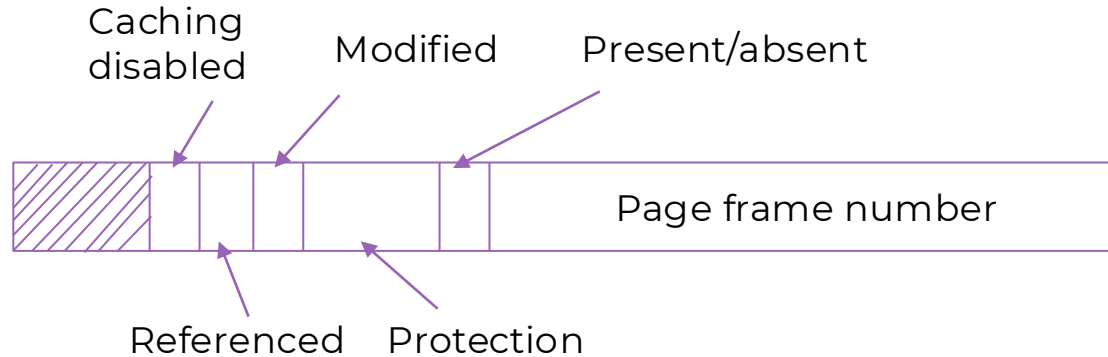
Page Tables

- The Memory Management Unit (MMU) has to maintain information about the **virtual->physical mapping**
- In the **simplest** case, this could be a simple array that stores the physical page number for each virtual page number
- The virtual address would then be split into two parts: an **index** into the mapping table, and then the **offset** within the page

The internal operation of the MMU with 16 4-KB pages.



Structure of a Page Table Entry (PTE)

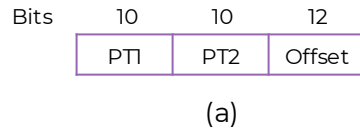


- Modified – has this page been written to?
 - If so, we will need to write to disk before evicting
- Referenced – has anyone used this page?
- Caching disabled – used if physical page is used for device I/O

Real Page Tables

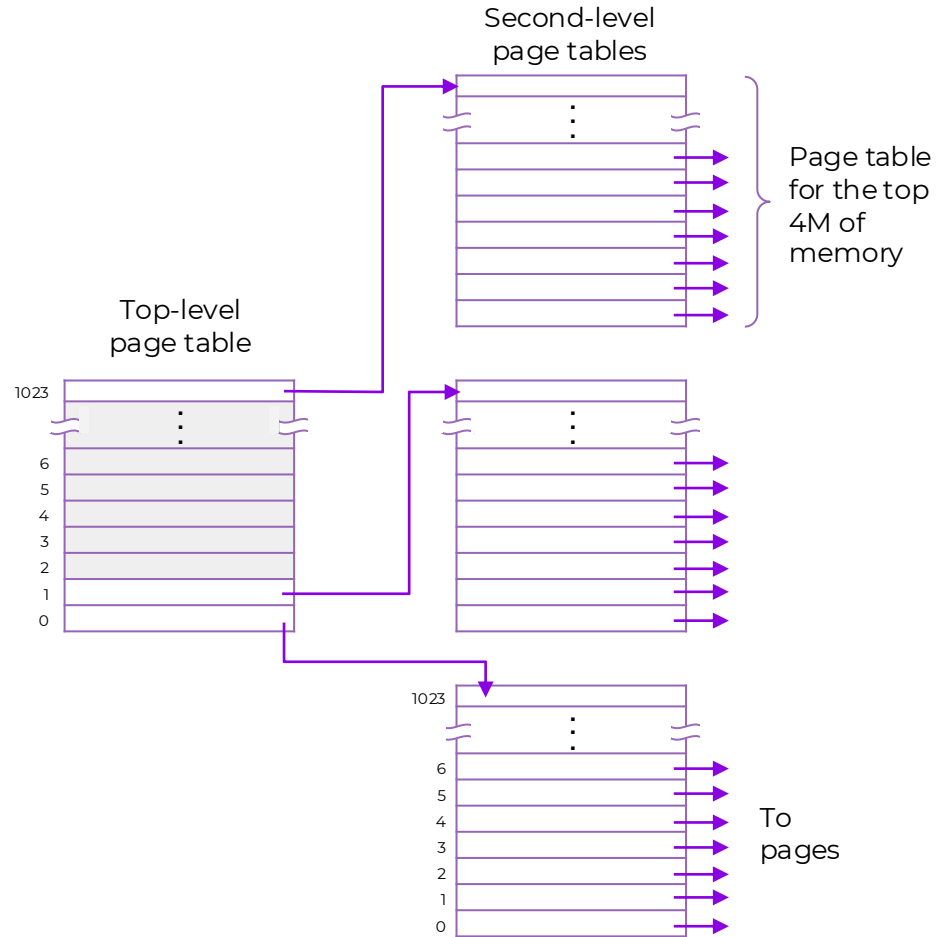
- In reality, this is very inefficient if the virtual address space is expected to be sparse (not many mapped pages)
- Instead, *multi-level* page tables are used
- The virtual address now has multiple indexes
- This allows us to only allocate tables for portions of the space that are used
- Remember: the page tables themselves are stored in memory!

Multilevel Page Tables



(a) A 32-bit address with two page table fields.

(b) Two-level page tables.



Protection

- Because the OS can give processes different virtual address spaces, we have already *solved* the problem of *isolation*
- But we may want to *protect processes* from *themselves* in some cases:
 - Detect *programmer errors* before they do damage
 - *Prevent attacks* that exploit software vulnerabilities

Translation Lookaside Buffer

- Walking the page table hierarchy each time memory is accessed gets very expensive
 - If we have to do 2 table lookups for every memory access, we've just made memory 3x slower
- Instead, the CPU keeps a *small* set of mappings that it can translate directly without consulting the page tables

Translation Lookaside Buffers

Valid	Virtual page	Modified	Protection	Page frame
1	140	1	RW	31
1	20	0	R X	38
1	130	1	RW	29
1	129	1	RW	62
1	19	0	R X	50
1	21	0	R X	45
1	860	1	RW	14
1	861	1	RW	75

TLB Misses

Soft:

- The page referenced is not in the TLB but is in memory.
- 10-20 Machine instructions (2 nanosecs)

Hard:

- Page is NOT in memory.
- Takes a few milliseconds (million times slower !!)

Software TLB Management

- In some architectures (e.g. SPARC, MIPS), the TLB is managed by software
- TLB entries are explicitly loaded by the OS
- If we look up an address and it's not in the TLB (a TLB miss), we raise a TLB fault
- The OS then has to fill in the missing TLB entry

Software TLB Management

- Why manage the TLB explicitly instead of letting hardware do it?
 - The MMU can be much simpler, which saves space on the CPU that can be used for more caches
 - Flexibility – the OS can choose its own algorithms for which TLB entries to evict and add
- But: this is generally slower than hardware-managed TLBs

TLBs and Context Switching

- TLBs map a virtual address to a physical one
- But once we change to a new process, these mappings are no longer valid, and *a TLB flush occurs*
- This makes *context switching more expensive* – the first few memory accesses a process makes will have to be serviced by walking the page tables

Tagged TLBs

- On some architectures, each TLB entry can be associated with a *tag* that says what address space it belongs to
- Now we don't have to flush the TLB when switching address spaces
- This can help make context switching faster – some TLB entries might still be valid when we switch back to a process

Page Table Overhead Calculations

- Using page tables has some overhead
- Just how much?
- Depends on the exact paging scheme used

Page Table Overhead Calculations

- "Worst" case:
 - single-level page table, 64-bit virtual address space, 4KB pages, 4 byte PTEs
 - $2^{64} / 4096 * 4 \text{ bytes} = 2^{52} * 2^2 = 2^{54} \text{ bytes} = 16 \text{ petabytes}$ of memory used for pages
 - 2^{50} = Petabyte or 1024 terabytes or million gigabytes
 - And that's just for **one process**!

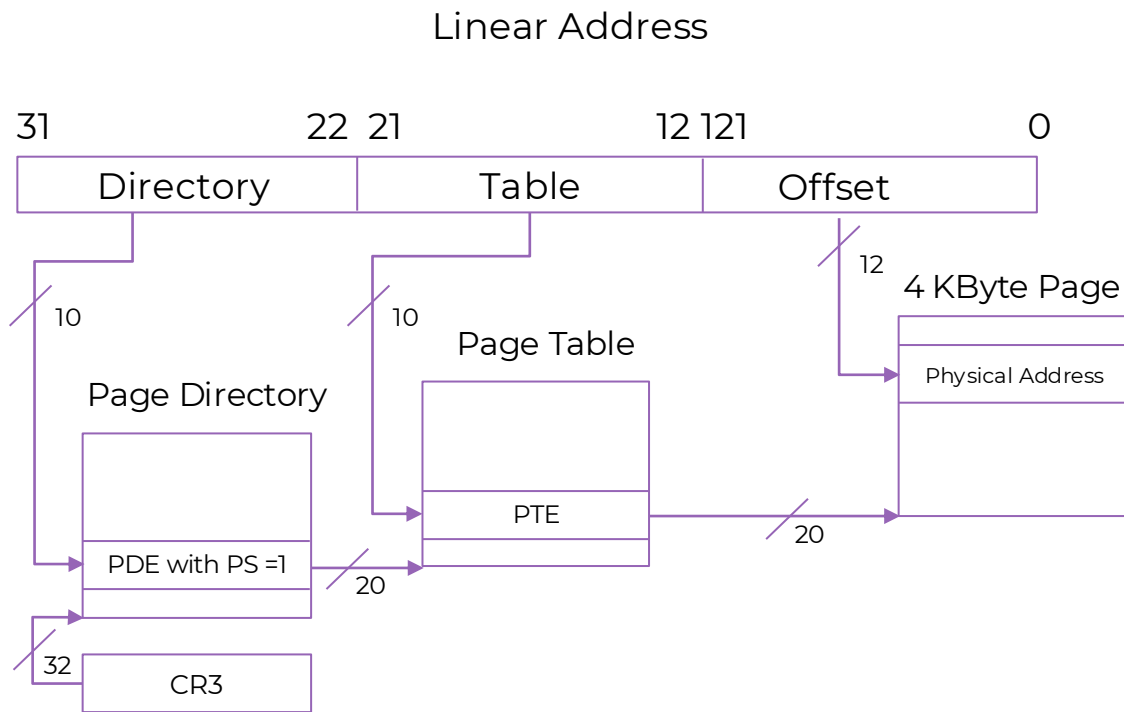
Virtual Address Translation in x86

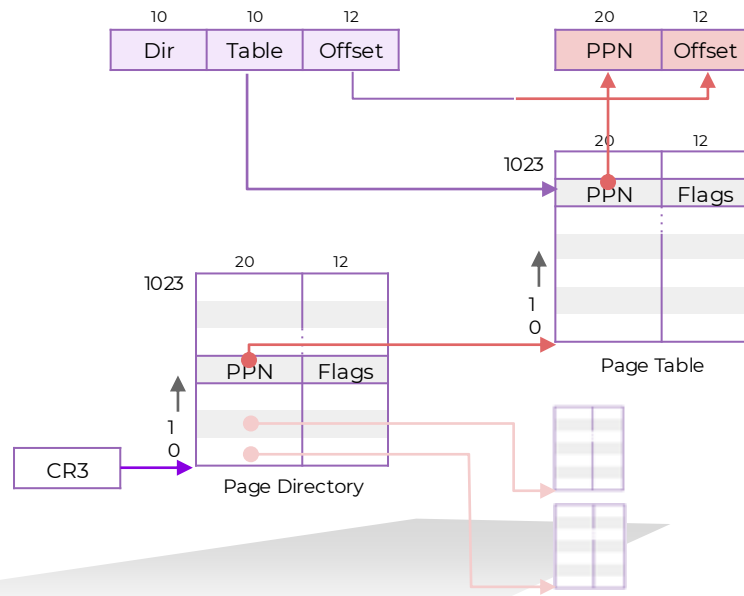
- We will, for now, consider only:
 - 32-bit x86
 - 4KB and 4MB ("super") pages
- So not:
 - 32-bit PAE (allows using more than 4GB of ram on a 32-bit machine)
 - 64-bit mode

x86 Paging Basics

- x86 virtual address translation uses a two-level page table:
 - A top-level *page directory stores pointers* to the page tables
 - *Page tables* contain the actual page table entries (PTEs) referring to physical page frames
- The current mappings in use are determined by the value of the *CR3 CPU register*, which stores the physical address of the page directory

x86 Paging





Page table and page directory entries are identical except for the D bit.

- P - Present
- W - Writable
- U - User
- WT - 1=write-through, 0=Write-back
- CD - Cache Disabled
- A - Accessed
- D - Dirty (0 in page directory)
- AVL - Available for system use

x86 PDEs and PTEs

- The entries in a page directory and a page table have an almost identical format
- Each 32-bit
- The basic structure:
 - Physical address of a page table (for PDEs) or memory page (for PTEs)
 - Protection, caching, etc. flags
 - A bit to indicate present/not present